

BD71, Big Data Spring 26

Hadoop Hive



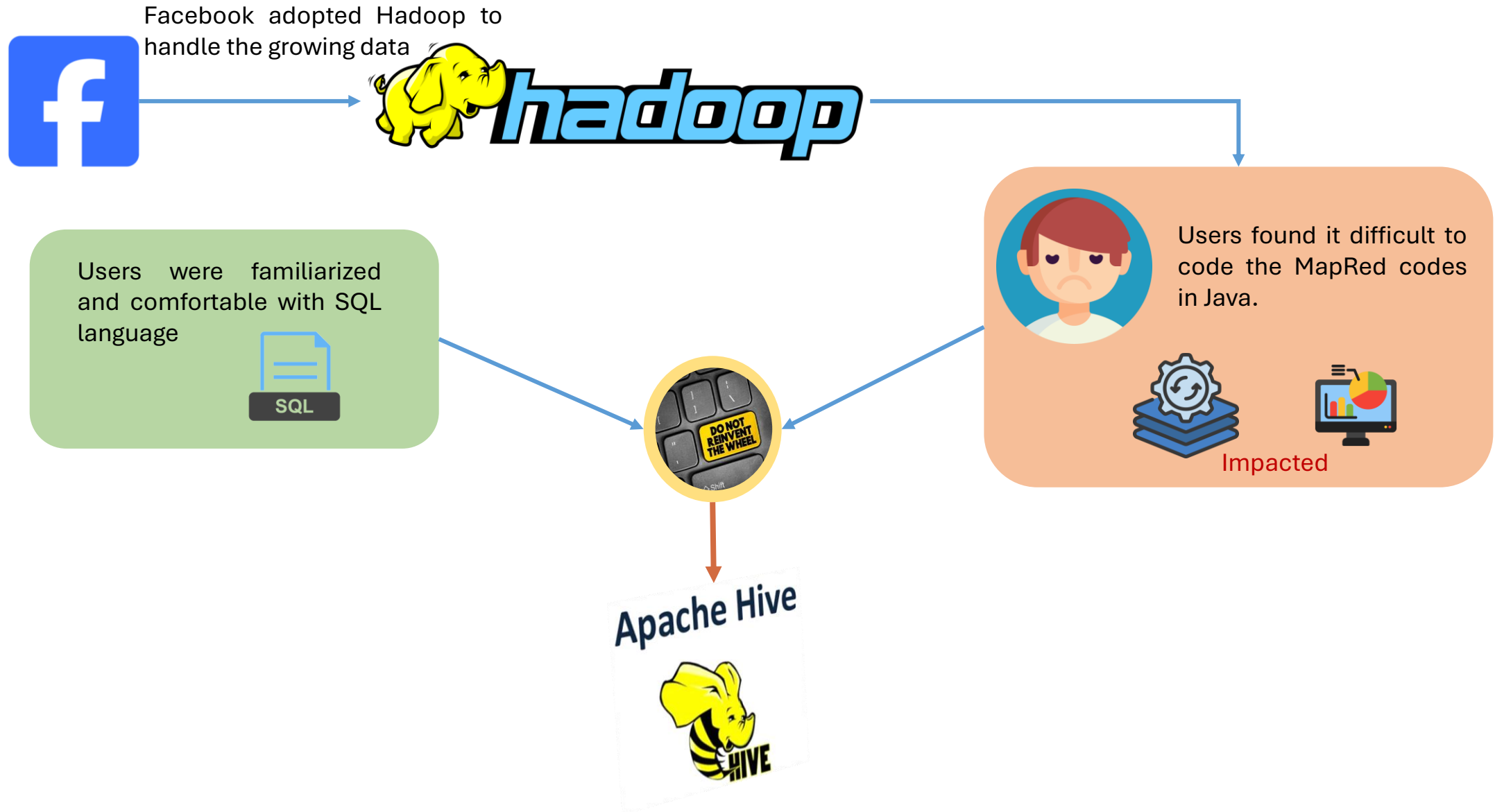
Prepared by
Pr. Mohamed KAS

Remote, 14 – 04 – 2026





Hive





Hive

Apache Hive is an open-source data warehouse for Hadoop. A data warehouse functions as a central repository where information comes from one or multiple data sources. It collects data from various heterogeneous sources with the main purpose of supporting analysis, querying with a language similar in syntax to SQL, and thereby facilitating the decision-making process.



Apache Hive is a Data Warehouse software originally created by Facebook. It allows for easy and fast execution of "SQL-like" queries to efficiently extract data from Apache Hadoop. Unlike Hadoop, Hive enables SQL queries to be executed without the need to write in Java.



Hive

Many companies across various industries use Apache Hive as part of their big data infrastructure for data storage, processing, and analysis. Some notable companies that use Hive include:



NETFLIX





Hive

Apache Hive



Tables are
partitioned &
bucketed

Easy to plug-in
custom MapRed
codes

Extensible:
Types, Formats,
Functions,
Scripts

Schema flexibility
& evolution

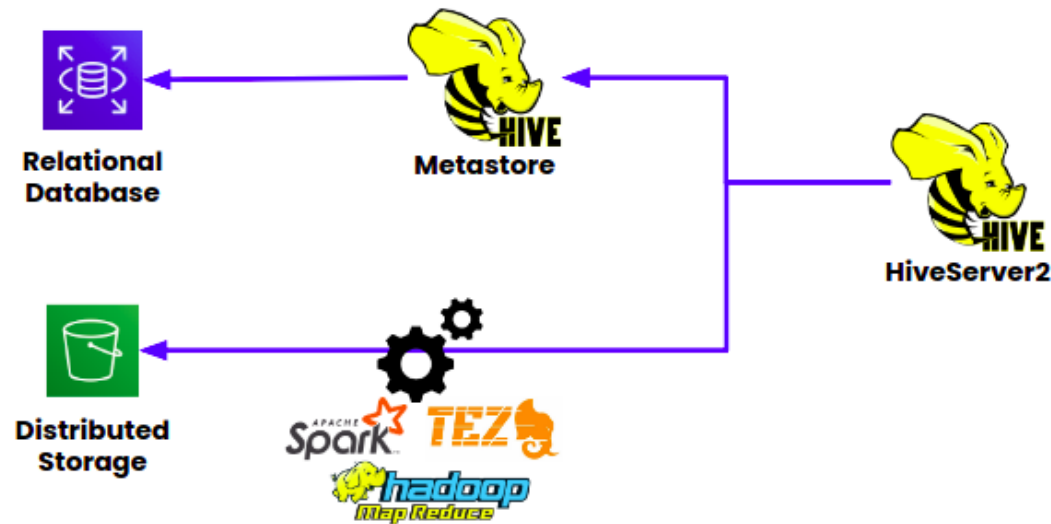
Hive table can be
defined directly
on HDFS



Hive

Hive is a system that maintains metadata describing the data stored in HDFS. It uses a relational database called Metastore (Derby by default) to ensure the persistence of metadata. Thus, a table in Hive is essentially composed of:

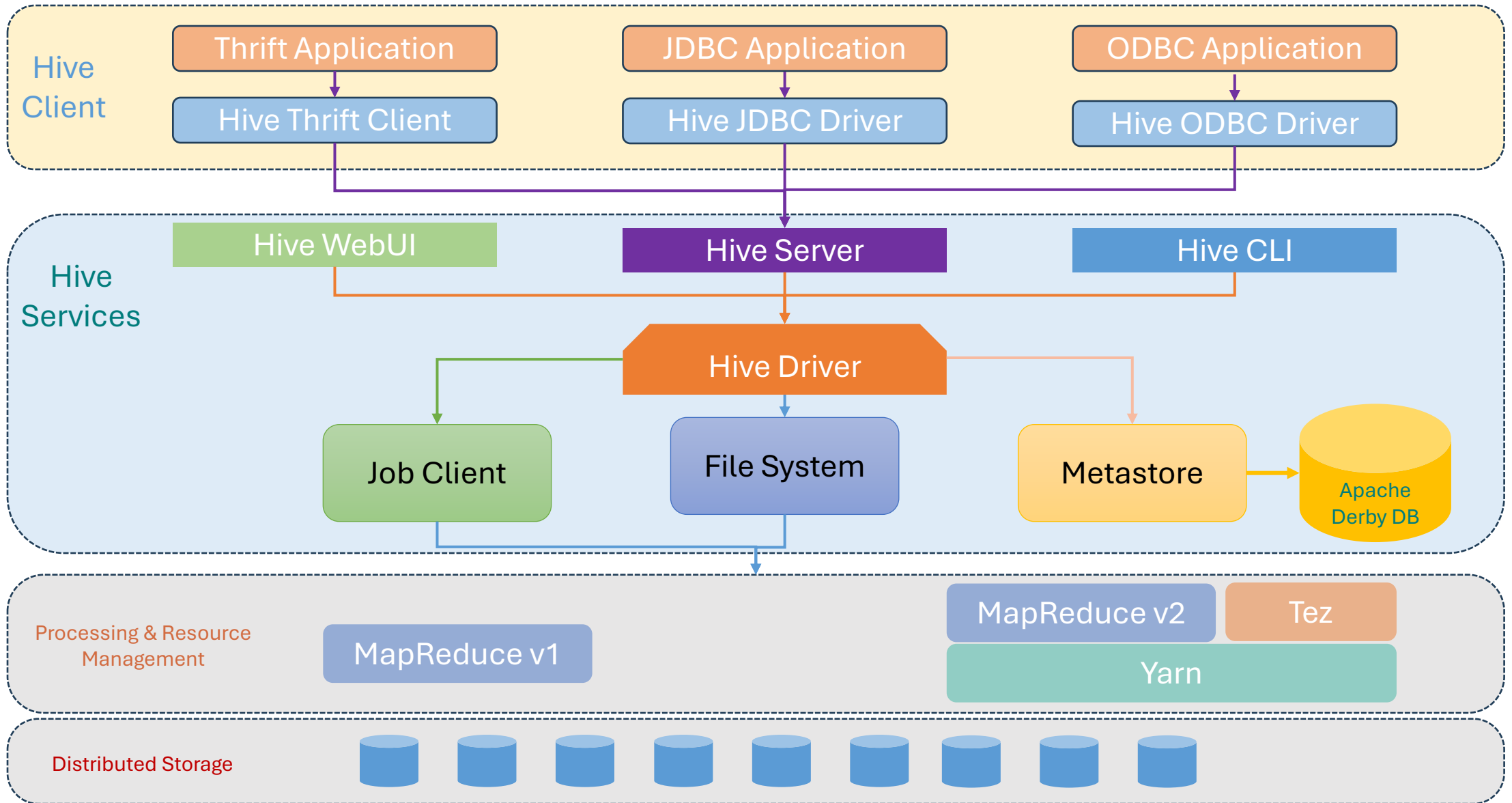
- A schema stored in the Metastore
- Data stored in HDFS



- ❑ With the Metastore data, Hive allows manipulating data as if they were persisted in tables (in the sense of a classic database management system) and querying them with its language HiveQL.
- ❑ Hive can convert HiveQL queries into MapReduce or Tez jobs (starting from version 0.13 of Hive, a HiveQL query can be translated into a job executable on Apache Tez, which is an execution framework on Hadoop that can replace MapReduce).



Hive

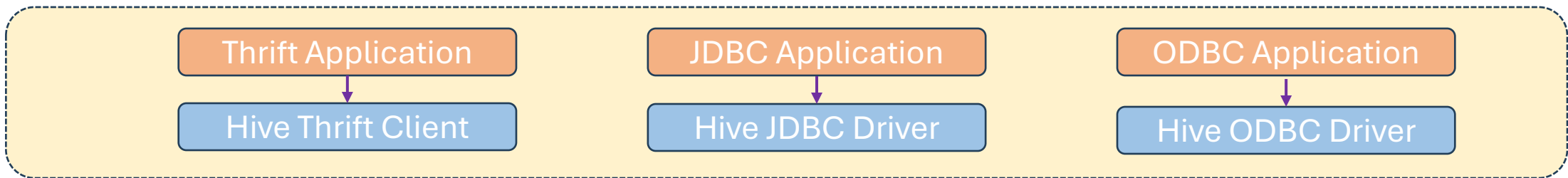




Hive

Hive Clients

- ❑ **Hive Thrift Client:** The Hive Thrift client supports various client applications using different programming languages to interact with Hive for data processing purposes. For direct integration of Map and Reduce functions as User Defined Functions (UDFs) in HiveQL queries, Hive relies on Apache Thrift, which allows writing UDFs in multiple programming languages (such as Java, Python, Ruby, etc.).
- ❑ **JDBC Application:** Any JDBC (Java DataBase Connectivity) application can connect to Hive using the Hive JDBC driver.
- ❑ **ODBC Application:** ODBC (Open DataBase Connectivity) clients can also connect to Hive using the Hive ODBC driver.





Hive

Hive Services

- ❑ **HiveServer/Thrift Server:** HiveServer is a service that allows a remote client to submit a query to Hive. The request can come from various programming languages such as Java, C++, Python. HiveServer is built on Apache Thrift, so we can also call it Thrift Server. Thrift is a software framework that serves requests from other programming languages that support Thrift. Various client applications such as Thrift Application, JDBC, ODBC can connect to Hive via HiveServer. All client requests are submitted to HiveServer only.
- ❑ **Hive CLI:** CLI (Command Line Interface) is the most commonly used interface to connect to Hive. Developers can submit their queries to Hive via Hive CLI. To access the Hive CLI, the client machine must have Hive installed. Once installed, you can access Hive by running "Hive" from the terminal.
- ❑ **Hive Web Interface (HWI):** The Hive Web Interface (HWI) is a graphical interface for submitting and executing Hive queries. To execute them via the web interface, you do not need to have Hive installed on your machine. A developer can open the URL in multiple windows to execute multiple queries in parallel. Another advantage of HWI is that you can browse the schema and tables of Hive.

Hiver WebUI

Hiver Server

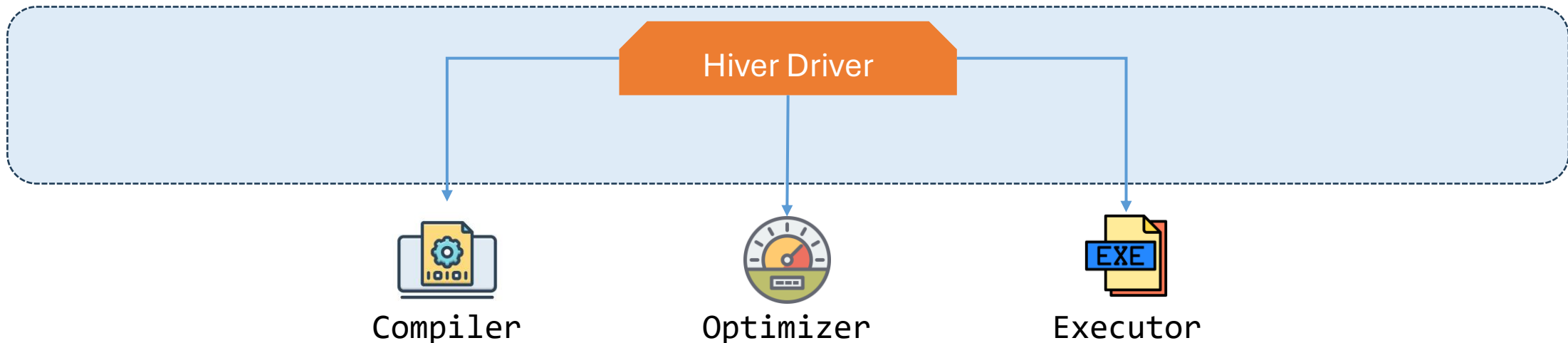
Hiver CLI



Hive

Hive Services

- ❑ **Hive Driver:** It is capable of receiving HQL queries from multiple sources such as Thrift, JDBC, and ODBS using the Hive server and directly from the Hive CLI and the web user interface. After receiving these queries, it transfers them to the compiler. The Hive driver consists of three different components:
 - ❑ **Compiler:** The Hive compiler is the one that analyzes the Hive query. It is responsible for semantic analysis and type checking. It generates the execution plan in the form of a Directed Acyclic Graph (DAG).
 - ❑ **Optimizer:** The Optimizer is responsible for performing transformation operations on the execution plan. It divides the task to improve efficiency and scalability. Thus, the optimizer will generate the optimized logical plan in the form of MR tasks. (HiveQL Process Engine = Compiler + Optimizer)
 - ❑ **Executor:** The execution engine executes the task after the compilation and optimization steps in the order of their dependencies using Hadoop. (Execution Engine = Executor)

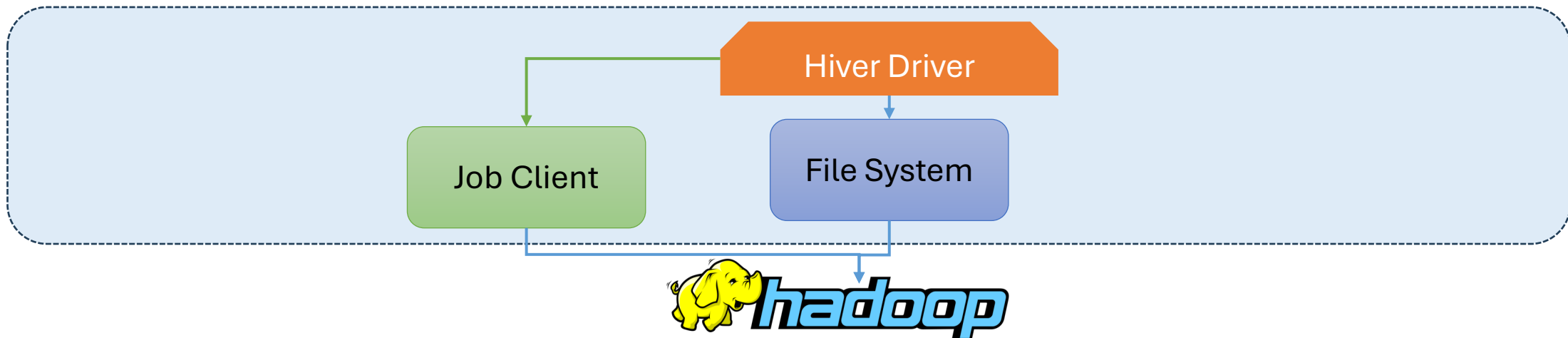




Hive

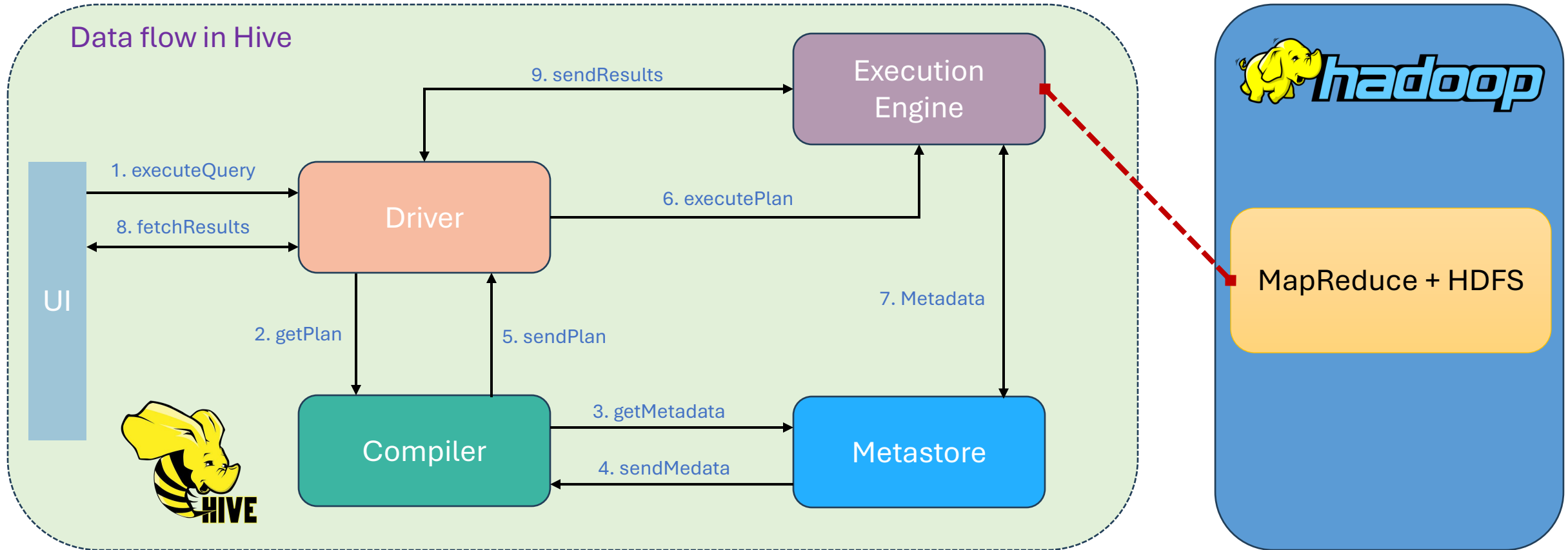
Hive Services

- ❑ **Hive Execution Engine:** The Execution Engine is responsible for executing the query plan generated by the Query Processor. It interacts with the underlying Hadoop infrastructure to schedule, monitor, and manage the execution of MapReduce, Tez, or other jobs.
- ❑ **Hive Storage Handler:** The Storage Handler is responsible for integrating Hive with different storage systems beyond HDFS, such as HBase, Amazon S3, or relational databases. It allows Hive to query data stored in these external systems using HiveQL.
- ❑ **Hive SerDe (Serializer/Deserializer):** SerDe is a serialization and deserialization library used to read and write data in various formats, such as delimited text, JSON, or Avro. It enables Hive to interpret the structure of data stored in files and convert it into Hive table rows and columns.





Hive





Hive

Hive Data Modelling

Table

Tables are created in Hive like RDBMS

Partitions

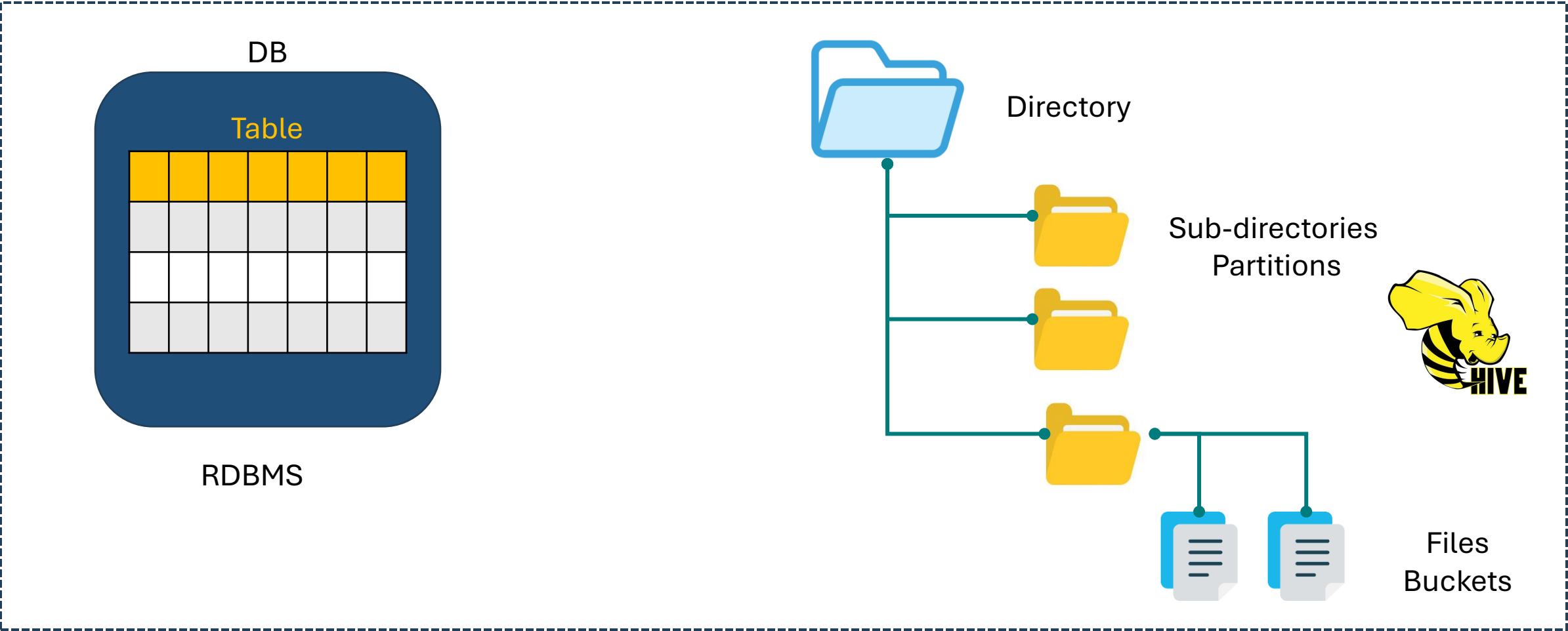
Grouping same type of data based on partition key

Buckets

Data present in partitions is further divided into buckets for efficient querying

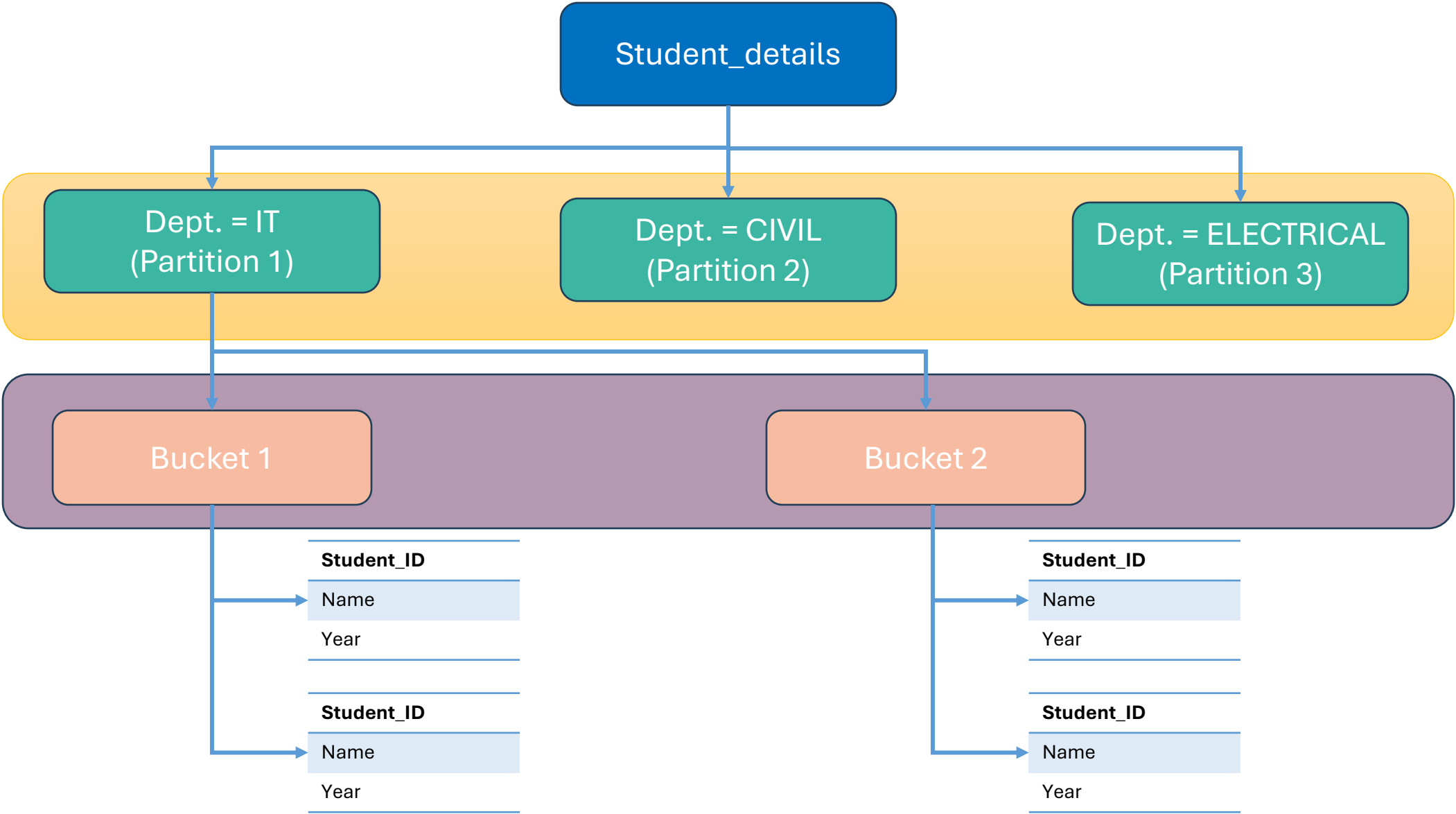


Hive Data Modelling





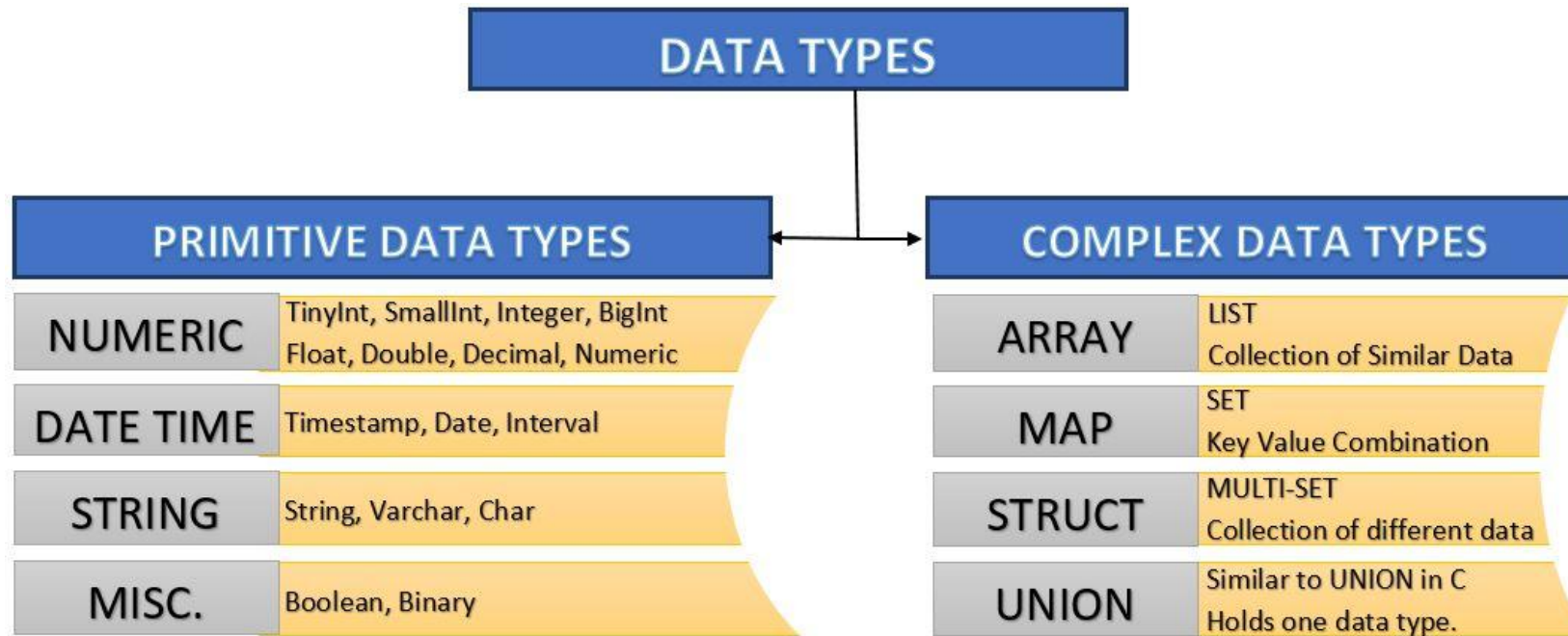
Hive





Hive

Hive Data Types





Hive

Hive HQL commands are:

☐ Data Definition Language (DDL):

Hive DDL commands are the instructions used to define and modify the structure of a table or a database in Hive. In other words, they are used to build or modify tables, databases, and other database objects.

For example, CREATE TABLE commands, DROP TABLE with extensions to define file formats, partitioning information, and bucketing.

☐ Data Manipulation Language (DML):

Hive's data manipulation language commands are used to insert, retrieve, modify, delete, and update data in Hive tables.

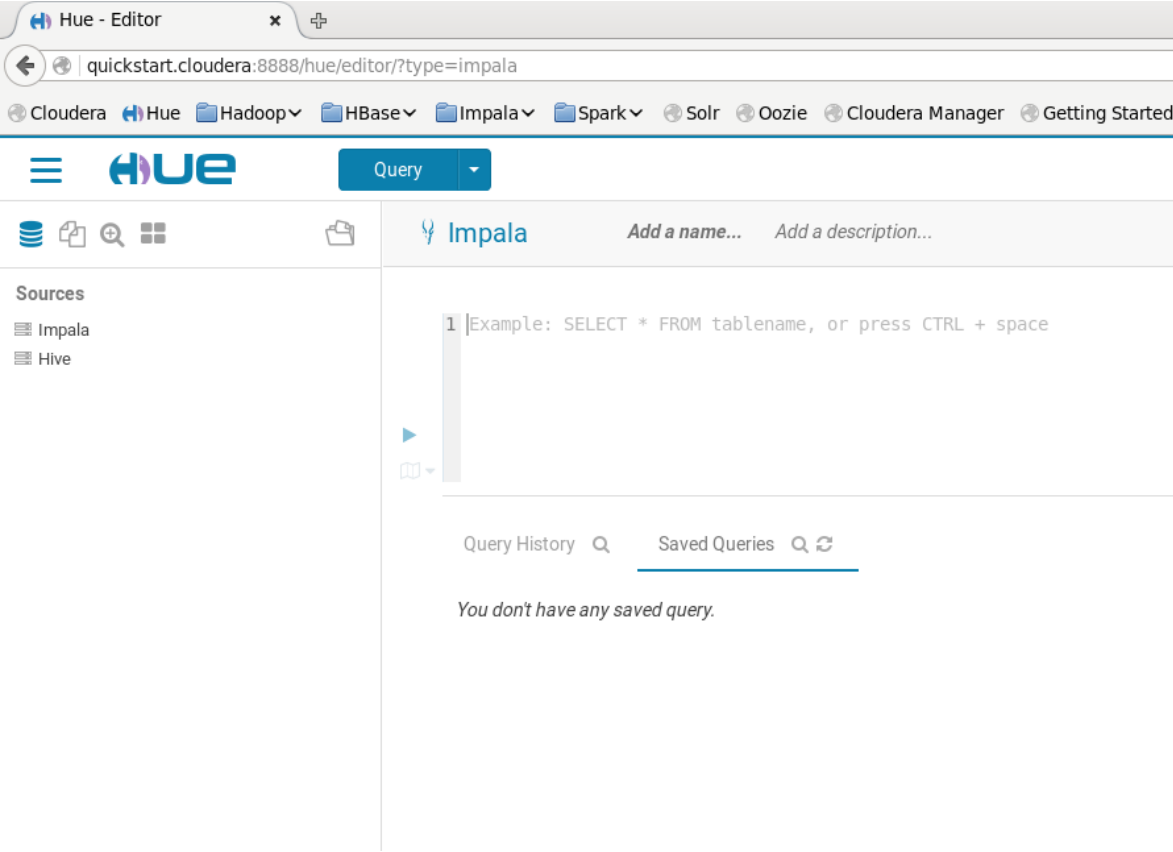
For example, loading data from external tables and inserting rows using the LOAD and INSERT commands.



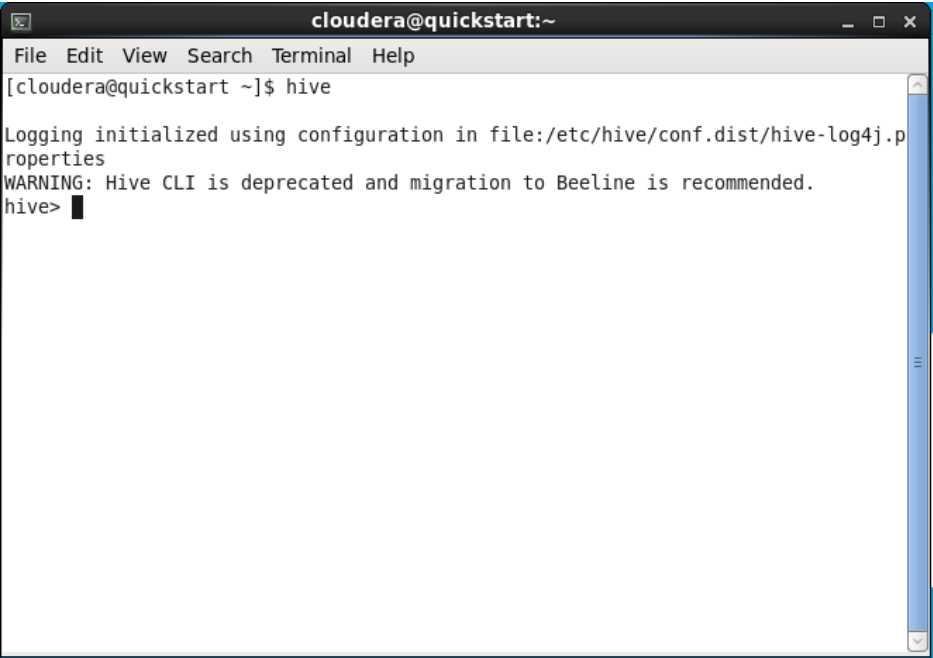
Hive

Hive UI

Cloudera HUE WebGUI



Hive CLI





Hive

HQL DDL Commands :

Command	Purpose
CREATE	Used to create a database or a table.
SHOW	Used to show the data of a given table and its properties.
ALTER	Used to modify the structure of a table or to edit the metadata associated to a database.
DESCRIBE	For tables, it describes the columns of a table. For databases, it displays the name of the database in Hive, its comment (if defined), and its location on the file system.
TRUNCATE	For tables, used to truncate and permanently delete rows from a table.
USE	The USE statement in Hive is used to select the specific database for a session on which all subsequent HiveQL statements will be executed.
DROP	For databases, it is used to delete the database. For tables, the "Drop Table" instruction deletes the data and metadata of an internal table. In the case of external tables, only the metadata is deleted.



Hive

HQL DDL Commands : Database

CREATE DATABASE/SCHEMA

```
CREATE DATABASE [IF NOT EXISTS] database_name [COMMENT database_comment] [LOCATION hdfs_path] [WITH DBPROPERTIES (property_name=property_value, ...)];
```

The screenshot shows a terminal window titled 'cloudera@quickstart:~' with the following output:

```
hive> CREATE DATABASE IF NOT EXISTS DS53 COMMENT "This the main DB of DS53 Lectures" LOCATION "/user/hive/warehouse/DS53" WITH DBPROPERTIES ("createdBy"="mkas", "version"="initial");
OK
Time taken: 0.174 seconds
hive>
```

Overlaid on the terminal is the Hue web interface. The left sidebar shows a file tree with 'warehouse' and 'DS53' (indicated by a blue arrow from the terminal output). The right pane shows the 'Impala' query editor with a text input field containing '1 Example:'.

```
ALTER DATABASE database_name SET LOCATION hdfs_path;
```



Hive

HQL DDL Commands : Database

```
USE DATABASE database_name;
```

```
DROP DATABASE [IF EXISTS] database_name [RESTRICT|CASCADE];
```

```
SHOW DATABASES;
```

```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
hive> SHOW DATABASES;  
OK  
default  
ds53  
Time taken: 0.008 seconds, Fetched: 2 row(s)  
hive> █
```

```
DESCRIBE DATABASE/SCHEMA [EXTENDED] database_name;
```

```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
hive> DESCRIBE DATABASE EXTENDED DS53;  
OK  
ds53    This the main DB of DS53 Lectures      hdfs://quickstart.cloudera:8020/user/hive/warehouse/DS53      cloudera      USER  
{createdBy=mkas, version=initial}  
Time taken: 0.35 seconds, Fetched: 1 row(s)  
hive> █
```



Hive

HQL DDL Commands : Tables

```
CREATE [EXTERNAL] TABLE [IF NOT EXISTS] table_name  
(column_name data_type [COMMENT column_comment], ...)  
[COMMENT table_comment]  
[PARTITIONED BY (column_name data_type, ...)]  
[CLUSTERED BY (column_names) [SORTED BY (column_names [ASC|DESC])] INTO num_buckets BUCKETS]  
[ROW FORMAT row_format]  
[STORED AS file_format]  
[LOCATION hdfs_path]  
[TBLPROPERTIES (property_name=property_value, ...)];
```

The screenshot displays the Hue web interface. On the left, a sidebar shows a file tree with a folder named 'ds53'. Inside 'ds53', there is a table named 'sales_data'. The table's schema is listed as follows:

- transaction_id (int)
- customer_id (int)
- transaction_total (decimal(10,2))
- transaction_date (date)

The main panel shows a terminal window titled 'cloudera@quickstart'. The terminal displays the following Hive command and its output:

```
hive> CREATE TABLE IF NOT EXISTS sales_data (  
  > transaction_id INT,  
  > customer_id INT,  
  > transaction_total DECIMAL(10,2)  
  >  
  > )  
  > COMMENT "A table to store sales transaction data"  
  > PARTITIONED BY(transaction_date DATE);  
OK  
Time taken: 0.1 seconds  
hive>
```

Hive

HQL DDL Commands : Tables

SHOW TABLE [IN database_name];

DROP TABLE [IF EXISTS] table_name;

TRUNCATE TABLE table_name [PARTITION partition_spec];

DESCRIBE [EXTENDED | FORMATTED] [database_name] table_name[.col_name ([.field_name]);

```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
hive> DESCRIBE EXTENDED sales_data;  
OK  
transaction_id      int  
customer_id         int  
transaction_total    decimal(10,2)  
transaction_date     date  
  
# Partition Information  
# col_name           data_type           comment  
transaction_date     date  
  
Detailed Table Information      Table(tableName=sales_data, dbName=ds53, owner:cloudera, createTime:1714428294, lastAccessTime:0, retention:0, sd:StorageDescriptor  
r(cols:[FieldSchema(name=transaction_id, type=int, comment:null), FieldSchema(name=customer_id, type=int, comment:null), FieldSchema(name=transaction_total, type:  
decimal(10,2), comment:null), FieldSchema(name=transaction_date, type=date, comment:null)], location:hdfs://quickstart.cloudera:8020/user/hive/warehouse/DS53/sale  
s_data, inputFormat:org.apache.hadoop.mapred.TextInputFormat, outputFormat:org.apache.hadoop.hive ql.io.HiveIgnoreKeyTextOutputFormat, compressed:false, numBucket  
s=1, serDeInfo:SerDeInfo(name=null, serializationLib:org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe, parameters:{serialization.format=1}), bucketCols:[], sor  
tCols:[], parameters:{}, skewedInfo:SkewedInfo(skewedColNames:[], skewedColValues:[], skewedColValueLocationMaps:{}), storedAsSubDirectories:false), partitionKeys  
:[FieldSchema(name=transaction_date, type=date, comment:null)], parameters:{numPartitions=0, transient_lastDdlTime=1714428294, comment=A table to store sales tran  
saction data), viewOriginalText:null, viewExpandedText:null, tableType:MANAGED_TABLE)  
Time taken: 0.056 seconds, Fetched: 11 row(s)  
hive>
```

```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
hive> DESCRIBE FORMATTED sales_data;  
OK  
# col_name           data_type           comment  
transaction_id      int  
customer_id         int  
transaction_total    decimal(10,2)  
  
# Partition Information  
# col_name           data_type           comment  
transaction_date     date  
  
# Detailed Table Information  
Database:            ds53  
Owner:               cloudera  
CreateTime:          Mon Apr 29 15:04:54 PDT 2024  
LastAccessTime:      UNKNOWN  
Protect Mode:        None  
Retention:           0  
Location:            hdfs://quickstart.cloudera:8020/user/hive/warehouse/DS53/sales_data  
Table Type:          MANAGED_TABLE  
Table Parameters:  
  comment            A table to store sales transaction data  
  numPartitions       0  
  transient_lastDdlTime 1714428294  
  
# Storage Information  
SerDe Library:       org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe  
InputFormat:         org.apache.hadoop.mapred.TextInputFormat  
OutputFormat:        org.apache.hadoop.hive ql.io.HiveIgnoreKeyTextOutputFormat  
Compressed:          No  
Num Buckets:         -1  
Bucket Columns:      []  
Sort Columns:        []  
Storage Desc Params: serialization.format 1  
Time taken: 0.051 seconds, Fetched: 35 row(s)  
hive>
```



Hive

HQL DDL Commands : Tables

The ALTER TABLE command in Hive is used to change the structure or properties of an existing table. This command supports a variety of operations that allow you to modify almost every aspect of a table. Here are some of the most common uses of the ALTER TABLE command in Hive:

```
ALTER TABLE table_name RENAME TO new_table_name;
```

```
ALTER TABLE table_name ADD COLUMNS (col_name data_type [COMMENT col_comment], ...);
```

```
ALTER TABLE table_name REPLACE COLUMNS (col_name data_type [COMMENT col_comment], ...);
```

```
ALTER TABLE table_name REPLACE COLUMNS (col_name data_type [COMMENT col_comment], ...);
```

```
ALTER TABLE table_name CHANGE [COLUMN] col_old_name col_new_name new_data_type [COMMENT col_comment] [FIRST|AFTER column_name];
```

```
ALTER TABLE table_name SET TBLPROPERTIES ('property_name' = 'property_value', ...);
```

```
ALTER TABLE table_name ADD [IF NOT EXISTS] PARTITION (partition_column = 'value', ...) [LOCATION 'location_path'];
```


Hive

HQL DML Commands

Command	Purpose
LOAD	Loads data into a Hive table from a file or directory in HDFS or from an external source.
SELECT	Retrieves data from one or more tables in Hive based on specified criteria.
INSERT	Adds new rows of data into a Hive table, either by specifying values directly or by selecting data from another table.
DELETE	Removes rows from a Hive table based on specified conditions.
UPDATE	Updates existing rows in a Hive table based on specified conditions.
EXPORT	Exports data from a Hive table into files in HDFS or into an external storage system.
IMPORT	Imports data from files in HDFS or from an external storage system into a Hive table.



Hive

HQL DML Commands

```
LOAD DATA [LOCAL] INPATH 'input_path' [OVERWRITE] INTO TABLE table_name [PARTITION (partition_column = 'value', ...)];
```

- LOCAL (optional): Specifies that the input path is local to the machine where the Hive metastore service is running. Omitting LOCAL assumes the input path is in HDFS.
- INPATH 'input_path': Specifies the path to the input data files or directory.
- OVERWRITE (optional): Specifies whether to overwrite existing data in the table. If omitted, the data is appended to the table.
- INTO TABLE table_name: Specifies the name of the Hive table where the data will be loaded.
- PARTITION (partition_column = 'value', ...): (optional) Specifies the partition column values for the data being loaded. This is used if the table is partitioned.



Hive

HQL DML Commands

```
SELECT [ALL|DISTINCT] select_expr, select_expr, ... FROM table_reference [WHERE where_condition  
[GROUP BY col_list] [HAVING having_condition] [ORDER BY col_list [ASC|DESC]]  
[LIMIT number_rows]
```

```
DELETE FROM table_name [WHERE condition];
```

```
UPDATE table_name SET column_name = value [, column_name = value ...] [WHERE condition];
```

```
EXPORT TABLE table_name TO 'export_path';
```

```
IMPORT TABLE table_name FROM 'import_path';
```